LMIS Forecasting Engine

AI-Enabled Labour Market Intelligence & Skill Demand–Supply Forecasting Engine

PROJECT HANDOFF / CONTEXT PACK FOR OTHER AI SYSTEMS

Purpose: transfer the project context, key decisions, constraints, architecture, and next steps to another AI assistant so work can continue without rebuilding the plan from scratch.

Important: This is a summary of planning decisions and recommendations, not proof that code, data integrations, or deployment already exist. The project's Master R&D PDF remains the source of truth. If this summary conflicts with it, flag the conflict and follow the R&D source.

1. Executive summary

Build a focused, data-first prototype that helps planners explore labour-market demand signals, qualified-supply estimates, skill gaps, evidence-limited forecasts, alerts, and training-seat scenarios. It is decision support—not an automatic policy engine.

- **Core flow:** import → validate → normalize occupations → calculate demand/supply/gap → forecast only when evidence permits → alerts → training-seat scenarios → export with caveats.
- **Recommended architecture:** modular monolith; React + TypeScript frontend; FastAPI backend; Pandas/NumPy; PostgreSQL target, with SQLite for local tests if useful.
- **Forecasting:** transparent baselines first. A complex model is retained only if it beats the baseline under time-based validation.
- **Demo:** reproducible, local-first, and able to run without venue internet. Synthetic data must be clearly labelled in the UI and exports.
- **Success:** a judge can trace a result to source status, period, geography, method, assumptions, and limitations.

2. Project identity

Working title: AI-Enabled Labour Market Intelligence and Skill Demand–Supply Forecasting Engine (LMIS).

Context: SIH 2026 problem space around labour-market intelligence and skill demand–supply forecasting. Verify the exact official problem statement, team metadata, and submission requirements from official team documents; do not infer them.

Intended users: policy/planning teams, skill-development planners, and analysts. Product posture: decision support, not automatic policy recommendations.

3. Scope

In scope for the first prototype

- CSV/XLSX import, schema validation, clear error messages, and import-run history.
- Occupation title normalization and mapping to a reference taxonomy; uncertain mappings require human review.
- Separate demand observations, qualified-supply estimates, training capacity/outcomes, and registration/context data.
- Documented gap and gap-rate calculations, including zero-demand and insufficient-evidence states.
- Baseline forecast with time-based evaluation; rule-based alerts; training-seat scenario calculator; export.
- Five screens: Overview, Occupation Explorer, Forecast & Alerts, Scenario Planner, Data & Reports.

Out of scope initially

- Unverified live government integrations, nationwide coverage claims, or automatic policy decisions.
- Treating registrations as available workers, training seats as current qualified supply, or postings as total vacancies.
- Deep learning for its own sake, LLM-first calculations, microservices, Kubernetes, vector databases, and queues without a proven need.
- Worker-level personal data unless official requirements and a clear privacy/legal basis justify it.

Stretch only after the core works: verified public connector, semantic occupation suggestions, maps, second forecast model, and additional reviewed languages.

4. Provisional assumptions — not confirmed

Item	Planning assumption	Action
Team	4–6 students; mixed skill levels	Confirm actual skills and owners.
Time/budget	3–4 weeks; ■0/near-zero	Confirm deadline and hosting limits.
Pilot	Maharashtra; Pune and Nashik	Proposal only; validate data fit.
Occupations	Solar installation, machine maintenance, computer operations	Illustrative set; validate taxonomy and evidence.
Data access	No compatible history or live API verified yet	Test before making claims.
Demo conditions	Ordinary laptops; internet may fail	Support local/offline demo.

5. Data integrity and sources

Investigate source families: job postings/employer signals; official labour-market statistics; training capacity, enrolment, completion and outcomes; worker-registration/context datasets; NCO occupation reference taxonomy.

Required status labels: REAL PUBLIC DATA, IMPORTED, SYNTHETIC DEMO, PLANNED. Use them on charts, tables, API results, and exports.

Non-negotiable: synthetic data is never official data; registrations are not qualified available workers; training seats are not current supply. Public source existence does not prove usable API access, suitable granularity, licensing, or permission.

Source references listed in the R&D

- NCS: ncs.gov.in
- PLFS Annual Report 2023–24: official PDF
- NCO-2015: dge.gov.in/nco-2015
- e-Shram: official portal
- NSQF: official framework page

These are starting points for verification, not confirmation that the project has working integrations.

6. Synthetic demo data

- Suggested shape: 36 monthly periods × 3 occupations × 2 districts = 216 occupation–district–month combinations per metric.
- Include controlled trend/seasonality; shortage, surplus and uncertain cases; missing values; duplicates; stale records; title aliases; and validation failures.
- Version the generator and dataset, record assumptions, and keep a README. All screens/exports must identify it as synthetic.
- Design imports so real data can later replace demo data without redesigning the core.

7. Metric definitions and guardrails

Metric	Definition	Guardrail
Demand	Documented demand signal from available hiring/posting evidence.	Not total vacancies or whole-market demand.
Qualified supply	Estimate of qualified people plausibly available in the target occupation/geography.	Not registrations, seats, or raw population.
Gap	Projected demand – projected qualified supply.	Only compare compatible unit, period, geography, definitions.
Gap rate	(Demand – supply) / demand × 100, only if demand > 0.	Undefined at zero demand; do not replace with zero.
Training capacity	Seats/capacity available for training.	Separate from current supply.
Scenario entrants	Additional seats × enrolment rate × completion rate × target-role/geography entry rate.	Show assumptions; avoid double counting.

Every result should carry period, geography, source status, update timestamp, method/version, coverage, and caveat. If evidence is insufficient, return an explicit state rather than fabricating an estimate.

8. Occupation mapping and forecasting

Occupation mapping

1. Preserve the raw title.
2. Normalize text without losing the original value.
3. Apply exact, verified codes/aliases first.
4. Optional semantic matching may suggest candidates but must not silently decide uncertain matches.
5. Send ambiguous/conflicting mappings for human review; record method and version.

Forecasting

1. Start with naïve and/or moving-average baselines.
2. Consider exponential smoothing only if sufficient, consistent history exists.
3. Use chronological/time-based validation and at minimum compare MAE to the baseline.
4. Keep a complex model only if it demonstrably beats baseline.
5. Show intervals only if the method supports them; otherwise state that uncertainty is not estimated.
6. Do not forecast when evidence is inadequate; explain why and what data is missing.

AI principle: optional semantic suggestions or justified forecasting may use AI/ML. Arithmetic, alert rules, audit trails, and scenarios should remain deterministic and testable.

9. Architecture and stack

Layer	Recommendation	Rule
Frontend	React + TypeScript; Recharts	Focused dashboard, reusable UI.
API	FastAPI	Python data ecosystem; explicit API contracts.
Processing	Pandas + NumPy	Imports, transformations, metrics.
Database	PostgreSQL target; SQLite for local tests if useful	Use migrations and explicit schemas.
Forecast	Baselines; statsmodels candidate; sklearn optional	Complexity must earn its place through evaluation.
Deployment	Local-first; Docker Compose if useful	Verify free-tier and venue constraints.
Background work	Synchronous initially	No Celery/Redis until demonstrated need.

Architecture: modular monolith. Avoid microservices, Kubernetes, vector databases, and LLM-based calculations in the first release.

10. Proposed entities and API

Logical entities: data_source, ingestion_run, raw_observation, geography, occupation, occupation_alias, demand_observation, training_record, supply_estimate, forecast_run, forecast_value, alert, scenario_run, audit_event.

Keep lineage metadata: source/status, import timestamp, source period, geography, unit, mapping method/version, validation state, and calculation/model version.

Method	Proposed endpoint	Purpose
GET	/api/v1/health	Health check.
GET	/api/v1/overview	Overview and source status.
GET	/api/v1/occupations	Occupation list/mapping status.
GET	/api/v1/gaps	Filtered gap metrics.
GET	/api/v1/forecasts	Forecast/evaluation or insufficient evidence.
GET	/api/v1/alerts	Evidence-backed alerts.
POST	/api/v1/imports	Import and validate CSV/XLSX.
GET	/api/v1/imports/{id}	Import status and errors.
POST	/api/v1/mappings/{id}/review	Human mapping decision.
POST	/api/v1/scenarios	Calculate a scenario.
GET	/api/v1/reports/{id}/export	Export a report.

These endpoints are proposed, not confirmed as implemented. Define schemas and test behavior before claiming they exist.

11. UX and demo

Views: Overview; Occupation Explorer; Forecast & Alerts; Scenario Planner; Data & Reports.

States: loading, empty data, import success/failure, unmapped occupation, stale data, insufficient evidence, forecast with limitations, export success/failure, offline/local mode.

Four-minute demo: show pilot/status labels → import → inspect occupation and gap → explain forecast/alert limitations → adjust scenario assumptions → export.

Make UI translation-ready, but ship additional languages only after human review. Demo must work from a clean checkout without internet.

12. Risks and minimum tests

Risk	Mitigation / evidence
No public API/data access	Verify early; file import and labelled synthetic fallback.
Synthetic mistaken for official	Persistent labels and export tests.
Incomparable demand/supply	Validate units, periods, geography and definitions.
Registration/seats treated as workers	Separate entities and tests.
Invalid forecast/leakage	Chronological validation, baseline comparison, insufficient-data state.
Wrong occupation mapping	Exact aliases, review queue, raw title and version retained.
Duplicate/stale records	Deduplication, timestamps, stale-data warnings.
Import/DB/demo failure	Automated tests, useful errors, local seed and offline rehearsal.
Scope overrun	Protect cut-line; defer stretch features.

Minimum test suite

- Gap/gap-rate arithmetic; zero demand; missing values; rounding; incompatible periods/geographies.
- CSV/XLSX valid and invalid schemas, missing columns, types, duplicates, stale/empty files.
- Mapping exact match, alias, ambiguous, unknown, and reviewed override.
- Forecast chronological split, baseline metric, insufficient history, and no future-data leakage.
- Scenario invalid/zero assumptions, visible assumptions, no negative or double-counted entrants.
- Exports preserve status labels, definitions, assumptions, periods, and caveats.
- End-to-end: clean setup → seed/import → dashboard → forecast state → scenario → export.

13. Four-week delivery plan

Week	Deliverables	Gate
1 Foundation	Lock scope/metrics; source inventory; repo/environment; schema/migrations; synthetic generator.	Dataset reproducible and labelled; data contract reviewed.
2 Data pipeline	CSV/XLSX validation; normalization/review; metrics/audit; first dashboard.	Actionable import errors; metric tests pass.
3 Intelligence	Baseline forecast/evaluation; insufficient-evidence state; alerts; scenarios; export.	Model compared to baseline; caveats visible.
4 Hardening	Tests/leakage checks; accessibility/responsive review; docs; demo rehearsal.	Clean-machine offline demo succeeds.

Never cut: source labels, definitions, import validation, gap tests, forecast caveats, scenario assumptions, export integrity, reproducible demo.

Cut first if behind: map view, second model, semantic matching, extra languages/sources, advanced alert ranking.

14. Suggested repository layout

```
lmis-prototype/
README.md | .env.example | docker-compose.yml
docs/ scope.md, metric-definitions.md, data-source-inventory.md,
architecture.md, model-evaluation.md, risk-register.md,
demo-runbook.md
data/ synthetic/ reference/ README.md
backend/ pyproject.toml
app/ main.py, api/, db/, models/, schemas/, config.py
app/services/ ingestion/, normalization/, occupation_mapping/,
metrics/, forecasting/, alerts/, scenarios/, audit/, exports/
tests/
frontend/ package.json
src/ api/, components/, pages/, charts/, forms/, i18n/
scripts/ generate_synthetic_data.py, validate_dataset.py, seed_demo.py
evidence/ import-tests/, forecast-evaluation/, demo-checklist.md
```

15. First ten implementation tasks

1. Confirm pilot, official requirements, definitions, and demo date.
2. Create repository, README, environment example, and run instructions.
3. Inventory sources: access, terms/license, fields, update frequency, geography, period, verification.
4. Define data contract and validation rules.
5. Generate versioned synthetic data with edge cases and provenance.
6. Create database schema/migrations and seed workflow.
7. Implement CSV/XLSX validation and import-run status.
8. Implement exact occupation normalization and review queue.
9. Test gap calculations, zero demand, missing inputs, and comparability.
10. Connect the Overview dashboard to the API with source status and caveats.

16. Open questions

- What are the actual team size, member skills, and task owners?
- What is the exact deadline and which deliverables are required?
- What official problem statement and team documents govern judging?
- Is there an existing repository/prototype? Inspect it before duplicating structure.
- Which sources/APIs are actually accessible and permitted? What history/granularity is available?
- Can demand and qualified supply be compared using common occupation, geography, period, and units?
- Is the proposed pilot appropriate given evidence and SIH constraints?
- What are the hosting, hardware, and venue-network constraints?
- Has the recommended stack been approved, or has the team already committed to another stack?

17. Instructions for the next AI assistant

- Treat the Master R&D PDF as primary source. If absent from the new conversation, ask the user to upload it; do not invent its contents.
- Separate confirmed facts, provisional assumptions, recommendations, and unknowns.
- Inspect the existing repository before coding; make focused changes and report exact files and tests.
- Never claim an API, dataset, feature, or deployment works without verifying it.
- Prioritize end-to-end correctness, traceability, source labels, and a reliable demo over extra technologies.
- Update documentation and tests when implementation changes.
- Ask only high-impact questions; otherwise proceed with explicitly labelled assumptions.

18. Decision log

Area	Current recommendation
Scope	Narrow pilot; no nationwide/live-data claims without proof.
Data	File import first; labelled, versioned synthetic fallback.
Mapping	Exact verified mapping first; human review for ambiguity.
Metrics	Deterministic and documented arithmetic.
Forecasting	Baseline first; complexity must beat baseline.
Architecture	Modular monolith; React/TypeScript + FastAPI + Pandas/NumPy.
Storage/demo	PostgreSQL target; local-first/offline demo.
Infrastructure	Synchronous initially; no queue/Kubernetes/microservices without need.
UX/delivery	Five screens, evidence labels, transparent uncertainty, export; four-week provisional plan.

19. Final principle

> The strongest demo is not ‘look how many technologies we used.’ It is: here is the evidence; here is how we transformed it; here is what the result means; here is what remains uncertain; and here is how a planner can test a decision without mistaking a scenario for a prediction.

End of handoff. Update this document as implementation decisions are verified; do not silently convert assumptions into facts.